IR OBJECT DETECTION & ALERT

Curiosity Kickoff

- Can we see IR light?
- How does IR detect objects?
- What happens when object comes near?
-

Concept Connection

An IR Object Detection & Alert system uses an infrared (IR) sensor to detect nearby objects. When an object is detected, the sensor sends a signal that triggers an alert such as an LED or buzzer, making it useful for obstacle detection, security, and automation applications.

1.IR (Infrared) Sensor :

Detects infrared light, which is invisible to human eyes.Often used for obstacle detection or remote control.

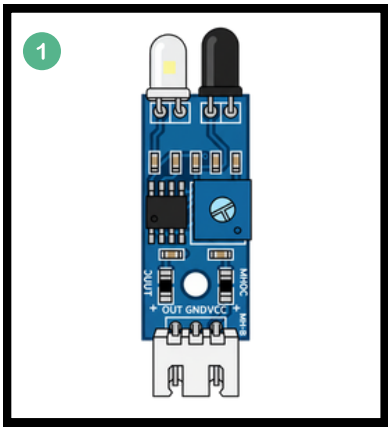
Pinout: 3 pins - VCC: Power, GND: Ground & OUT: Digital signal

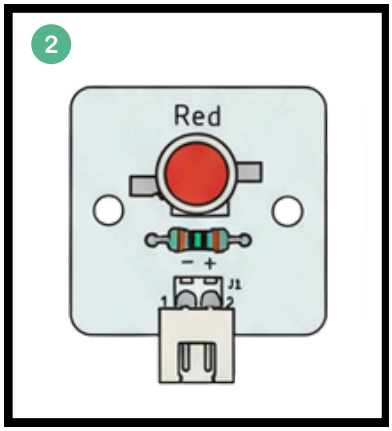
2. Watt SMD LED

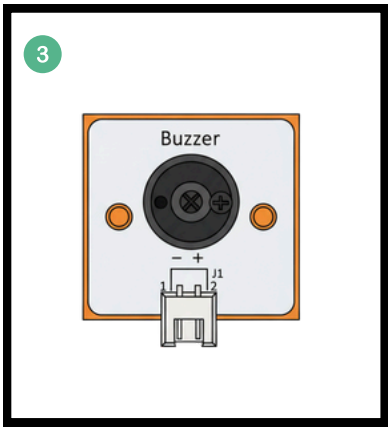
I'm flat and powerful. Just one of me can light up your whole table like a torch.I don't like wasting energy, so I shine super bright without getting too hot,You've already met me — I live in your mobile flashlight!

3.Buzzer

A sound-making device that works when electricity makes a thin metal plate vibrate rapidly inside. The vibration creates buzzing or beeping sounds. Some buzzers can even change frequency to play simple tunes or melodies.







Beginner: Object Watch

Class Explanation

An Object Watch using IR Sensor system uses an infrared sensor to continuously monitor the presence of objects. When an object comes within range, the sensor detects it and triggers an output like an LED or alarm, making it useful for security, monitoring, and automation applications.

Materials Required

- Arduino with attached shield
- 1 × IR sensor
- 1 × white 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

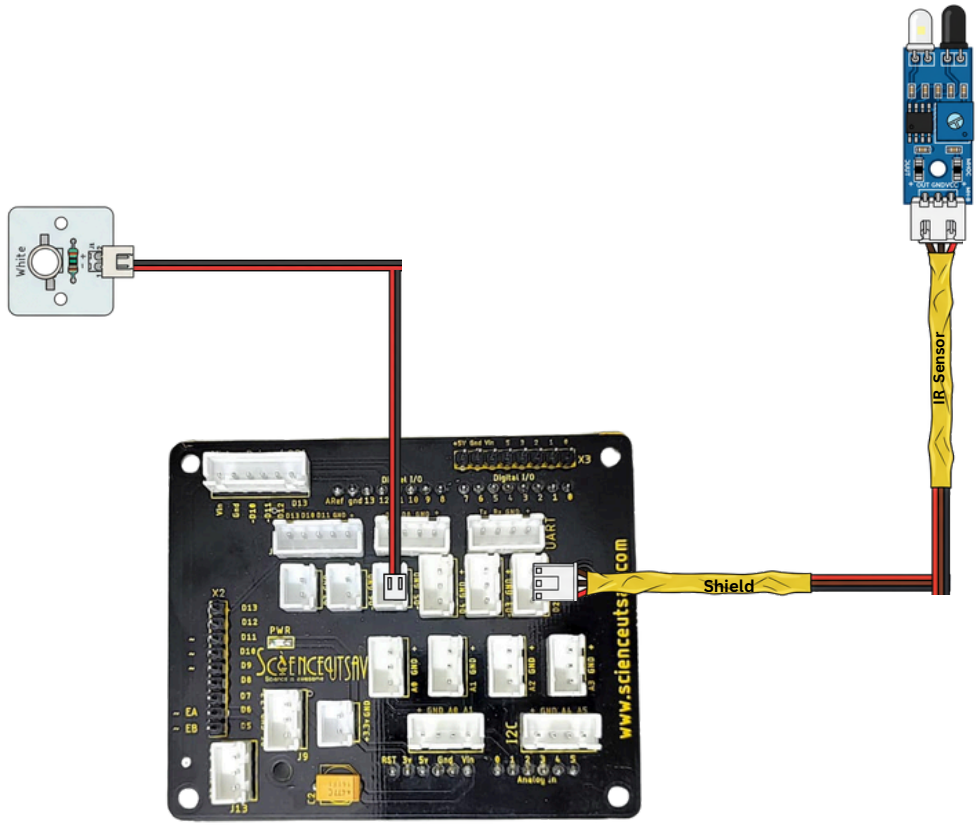
How to Connect

Use RMC wires to connect :

- 1 × IR sensor→ Arduino pin D2 and GND
- 1 × white 1 Watt SMD → Arduino pin D3 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define IR_PIN 2  // IR sensor connected to D2
#define LED_PIN 6  // LED connected to D3

void setup() {
 pinMode(IR_PIN, INPUT);
 pinMode(LED_PIN, OUTPUT);
 digitalWrite(LED_PIN, LOW);
}

void loop() {
 int IRState = digitalRead(IR_PIN);

 if (IRState == LOW) {
  digitalWrite(LED_PIN, HIGH); // Turn LED ON
 } else {
  digitalWrite(LED_PIN, LOW); // Turn LED OFF
 }
}
```

Code Explanation

- this Arduino program, an IR sensor is connected to pin 2 (IR_PIN) and an LED is connected to pin 3 (LED_PIN).
- The setup() function runs once when the Arduino starts. It sets the IR sensor as INPUT and the LED as OUTPUT using pinMode(). The LED is initially turned OFF using digitalWrite().
- The loop() function runs continuously. It reads the IR sensor value using digitalRead() and stores it in IRState.
- If the IR sensor detects an object (LOW signal), the LED turns ON. If no object is detected (HIGH), the LED turns OFF.

So overall, the LED glows when an object is detected by the IR sensor and turns OFF otherwise.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Object Watch Basics but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define IR_PIN   2    // IR sensor connected to D2
#define LED1_PIN 3    // LED1 on D3
#define LED2_PIN 4    // LED2 on D4

void setup() {
  pinMode(IR_PIN, INPUT);
  pinMode(LED1_PIN, OUTPUT);
  pinMode(LED2_PIN, OUTPUT);

  digitalWrite(LED1_PIN, LOW);
  digitalWrite(LED2_PIN, LOW);
}

void loop() {
  int IRState = digitalRead(IR_PIN);

  if (IRState == LOW) {
    digitalWrite(LED1_PIN, HIGH); // LED1 ON
    digitalWrite(LED2_PIN, HIGH); // LED2 ON
  } else {
    digitalWrite(LED1_PIN, LOW);  // LED1 OFF
    digitalWrite(LED2_PIN, LOW);  // LED2 OFF
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn

the basic idea of Object Watch Basics how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Intermediate: Obstacle Alert Guard

Class Explanation

In this class, students learn An Obstacle Alert Guard system uses sensors like infrared (IR) to detect obstacles in its path. When an object is detected, it triggers an alert such as an LED or buzzer, helping in safety, navigation, and automated protection systems.

Materials Required

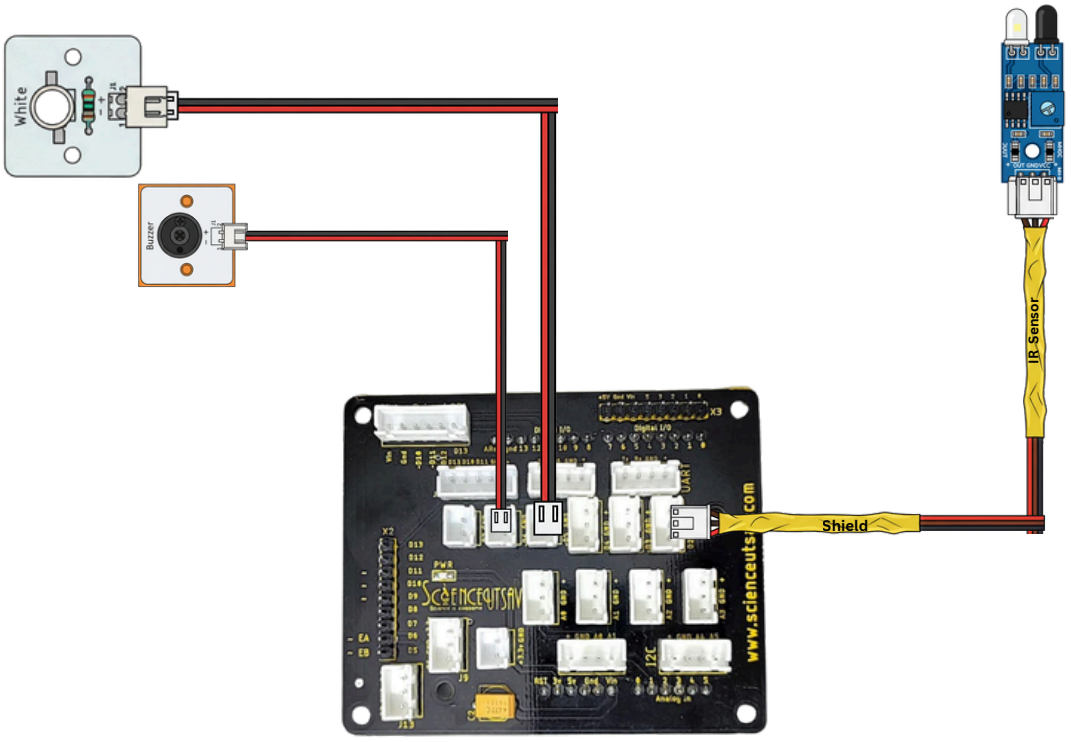
- Arduino with attached shield
- 1 × IR sensor
- 1 × white 1 Watt SMD LED on breakout board
- 1 × Buzzer
- RMC wires
- USB cable

How to Connect

- Use RMC wires to connect :
- 1 × IR sensor→ Arduino pin D2 and GND
- 1 × white 1 Watt SMD LED → Arduino pin D6 and GND
- 1 × Buzzer→ Arduino pin D5 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define IR_PIN 2  // IR sensor connected to D2
#define BUZZ_PIN 5 // Buzzer connected to D5
#define LED_PIN 6 // LED connected to D6 (works opposite to buzzer)

void setup() {
 pinMode(IR_PIN, INPUT);
 pinMode(BUZZ_PIN, OUTPUT);
 pinMode(LED_PIN, OUTPUT);
 digitalWrite(BUZZ_PIN, LOW);
 digitalWrite(LED_PIN, LOW);
}

void loop() {
 int IRState = digitalRead(IR_PIN);

 if (IRState == LOW) {
  digitalWrite(BUZZ_PIN, HIGH); // Turn buzzer ON
  digitalWrite(LED_PIN, LOW); // LED OFF (opposite behavior)
 } else {
  digitalWrite(BUZZ_PIN, LOW); // Turn buzzer OFF
  digitalWrite(LED_PIN, HIGH); // LED ON (opposite behavior)
 }
}
```

Code Explanation

- In this Arduino program, an IR sensor is connected to pin 2 (IR_PIN), a buzzer to pin 5 (BUZZ_PIN), and an LED to pin 6 (LED_PIN). The LED works in the opposite way to the buzzer.

- The setup() function runs once when the Arduino starts. It sets the IR sensor as INPUT and both the buzzer and LED as OUTPUT using pinMode(). Initially, both are turned OFF.

- The loop() function runs continuously. It reads the IR sensor value using digitalRead() and stores it in IRState.

- If an object is detected (LOW signal), the buzzer turns ON and the LED turns OFF. If no object is detected (HIGH), the buzzer turns OFF and the LED turns ON.

So overall, the system detects objects using the IR sensor and controls the buzzer and LED in opposite ways.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Obstacle Alert Guard with Arduino but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define IR_PIN 2     // IR sensor connected to D2

#define BUZZ1_PIN 5  // Buzzer 1 on D5
#define BUZZ2_PIN 4  // Buzzer 2 on D4

#define LED_PIN 6    // LED on D6 (opposite behavior)

void setup() {
  pinMode(IR_PIN, INPUT);

  pinMode(BUZZ1_PIN, OUTPUT);
  pinMode(BUZZ2_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);

  digitalWrite(BUZZ1_PIN, LOW);
  digitalWrite(BUZZ2_PIN, LOW);
  digitalWrite(LED_PIN, LOW);
}

void loop() {
  int IRState = digitalRead(IR_PIN);

  if (IRState == LOW) {
    digitalWrite(BUZZ1_PIN, HIGH); // Buzzer 1 ON
    digitalWrite(BUZZ2_PIN, HIGH); // Buzzer 2 ON
    digitalWrite(LED_PIN, LOW);    // LED OFF
  } else {
    digitalWrite(BUZZ1_PIN, LOW);  // Buzzer 1 OFF
    digitalWrite(BUZZ2_PIN, LOW);  // Buzzer 2 OFF
    digitalWrite(LED_PIN, HIGH);   // LED ON
  }
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of Obstacle Alert Guard with Arduino how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Advanced: IR Obstacle Detector

Class Explanation

In this class, students learn the An IR Obstacle Detector uses an infrared sensor to detect objects in its path. It emits IR signals and senses their reflection from nearby objects. When an obstacle is detected, it triggers an output like an LED or buzzer, useful in robotics, safety, and automation systems.

Materials Required

- Arduino with attached shield
- 1 × IR sensor
- 1 × White 1 Watt SMD LED on breakout board
- 1 × Buzzer
- RMC wires
- USB cable

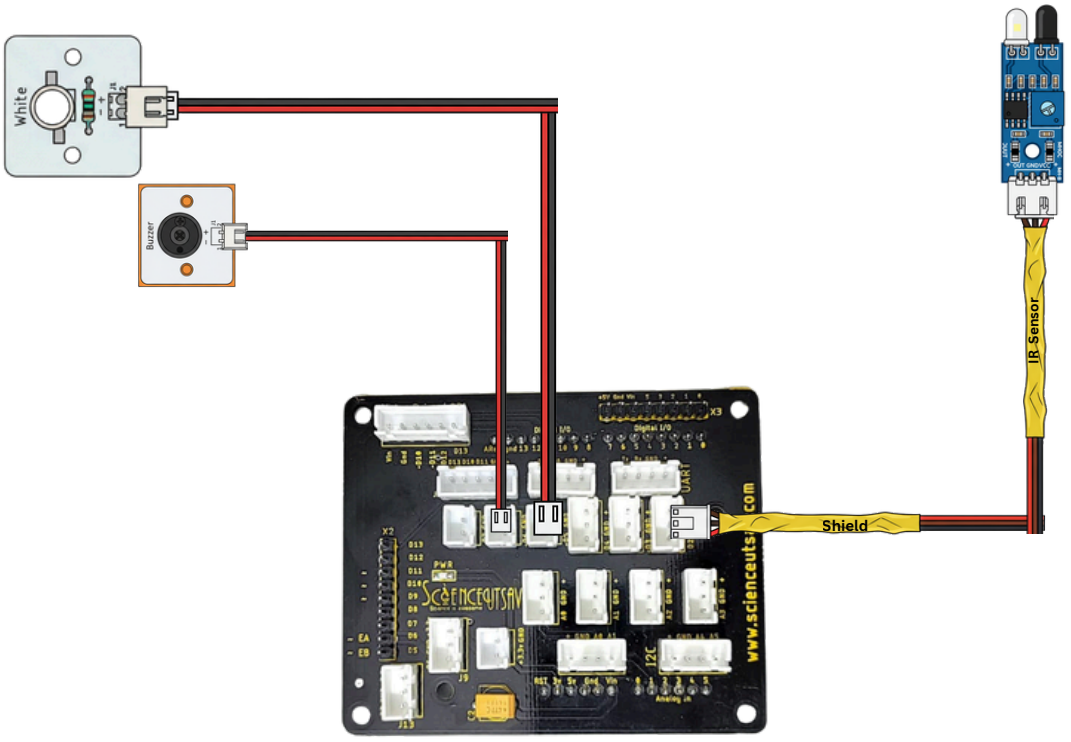
How to Connect

Use RMC wires to connect :

- 1 × IR sensor → Arduino pin D2 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D5 and GND
- 1 × Buzzer → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
// IR LED Detector using two LEDs on D5 (detected) and D6 (idle)
// Board: Arduino UNO (or compatible)

#define IR_PIN 2     // IR sensor connected to D2
#define LED_DETECTED 5  // LED that turns ON when something is detected
#define LED_IDLE 6    // LED that stays ON when nothing is detected

void setup() {
 pinMode(IR_PIN, INPUT);    // Use INPUT for modules with onboard comparator
 pinMode(LED_DETECTED, OUTPUT);
 pinMode(LED_IDLE, OUTPUT);

 // Initial states: assume no detection
 digitalWrite(LED_DETECTED, LOW);
 digitalWrite(LED_IDLE, HIGH);
}

void loop() {
 int irState = digitalRead(IR_PIN);

 // Many IR obstacle sensors pull the output LOW when an object is detected.
 // If your module behaves opposite, swap HIGH/LOW in the condition below.
 bool detected = (irState == LOW);

 if (detected) {
  digitalWrite(LED_DETECTED, HIGH); // Turn ON D5 LED (detected)
  digitalWrite(LED_IDLE, LOW);    // Turn OFF D6 LED
 } else {
  digitalWrite(LED_DETECTED, LOW);  // Turn OFF D5 LED
  digitalWrite(LED_IDLE, HIGH);   // Turn ON D6 LED (idle/clear)
 }
}
```

Code Explanation

- In this Arduino program, an IR sensor is connected to pin 2 (IR_PIN), and two LEDs are connected to pins 5 (LED_DETECTED) and 6 (LED_IDLE). One LED shows detection, and the other shows idle status.

- The setup() function runs once when the Arduino starts. It sets the IR sensor as INPUT and both LEDs as OUTPUT using pinMode(). Initially, the detected LED is OFF and the idle LED is ON.

- The loop() function runs continuously. It reads the IR sensor value using digitalRead() and stores it in irState

.

- If an object is detected (LOW signal), the LED on pin 5 turns ON and the LED on pin 6 turns OFF. If no object is detected (HIGH), the LED on pin 5 turns OFF and the LED on pin 6 turns ON.

So overall, one LED indicates object detection while the other shows when the path is clear.

Try This Yourself Code

This Try This Yourself code keeps the same topic of IR Obstacle Detector but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define IR_PIN 2          // IR sensor connected to D2
#define LED_DETECTED 5    // LED for detection
#define LED_IDLE 6        // LED for idle state
#define LED_EXTRA 7       // Extra LED (same as detected)

void setup() {
  pinMode(IR_PIN, INPUT);
  pinMode(LED_DETECTED, OUTPUT);
  pinMode(LED_IDLE, OUTPUT);
  pinMode(LED_EXTRA, OUTPUT);

  // Initial state
  digitalWrite(LED_DETECTED, LOW);
  digitalWrite(LED_IDLE, HIGH);
  digitalWrite(LED_EXTRA, LOW);
}

void loop() {
  int irState = digitalRead(IR_PIN);
  bool detected = (irState == LOW);

  if (detected) {
    digitalWrite(LED_DETECTED, HIGH); // Detected LED ON
    digitalWrite(LED_EXTRA, HIGH);    // Extra LED ON
    digitalWrite(LED_IDLE, LOW);      // Idle LED OFF
  } else {
    digitalWrite(LED_DETECTED, LOW);  // Detected LED OFF
    digitalWrite(LED_EXTRA, LOW);     // Extra LED OFF
    digitalWrite(LED_IDLE, HIGH);     // Idle LED ON
  }
}
```

Obective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learnctive:

the basic idea of IR Obstacle Detector how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Expert: Intelligent Object Monitor

Class Explanation

An Intelligent Object Monitor uses sensors like infrared (IR) to detect and track the presence of objects. It processes the sensor signals to control outputs such as LEDs or alarms, enabling smart monitoring, security, and automation in various electronic applications.

Materials Required

- Arduino with attached shield
- 1 × IR sensor
- 4 × White 1 Watt SMD LED on breakout board
- RMC wires
- USB cable

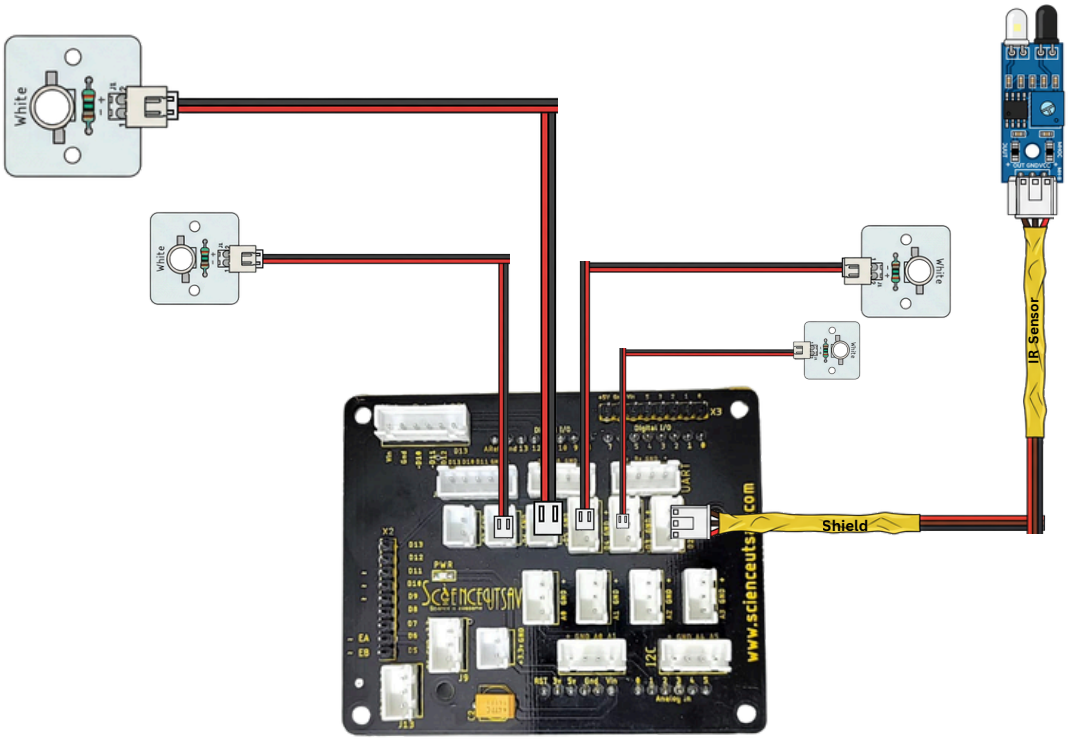
How to Connect

Use RMC wires to connect :

- 1 × IR sensor → Arduino pin D2 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D3 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D4 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D5 and GND
- 1 × White 1 Watt SMD LED → Arduino pin D6 and GND

Then connect the Arduino to your computer with the USB cable.

“Refer to the link or QR at the back of the book to get the code file for this project.”



Code

```
#define IR_PIN 2     // IR sensor on D2

#define LED1 3       // LED1
#define LED2 4       // LED2
#define LED3 5       // LED3
#define LED4 6       // LED4

int count = 0;
int lastIRState = HIGH;

void setup() {
  pinMode(IR_PIN, INPUT);

  pinMode(LED1, OUTPUT);
  pinMode(LED2, OUTPUT);
  pinMode(LED3, OUTPUT);
  pinMode(LED4, OUTPUT);

  // All OFF initially
  digitalWrite(LED1, LOW);
  digitalWrite(LED2, LOW);
  digitalWrite(LED3, LOW);
  digitalWrite(LED4, LOW);
}

void loop() {
  int irState = digitalRead(IR_PIN);

  // Detect object (LOW = detected)
  if (irState == LOW && lastIRState == HIGH) {
    count++;

    // Control LEDs based on count
    if (count == 1) {
      digitalWrite(LED1, HIGH);
    }
    else if (count == 2) {
```

```
 digitalWrite(LED2, HIGH);
 }
 else if (count == 3) {
 digitalWrite(LED3, HIGH);
 }
 else if (count == 4) {
 digitalWrite(LED4, HIGH);
 }
 else {
 // Reset everything
 digitalWrite(LED1, LOW);
 digitalWrite(LED2, LOW);
 digitalWrite(LED3, LOW);
 digitalWrite(LED4, LOW);
 count = 0;
 }

 delay(200); // debounce
 }

 lastIRState = irState;
}
```

Code Explanation

- In this Arduino program, an IR sensor is connected to pin 2 (IR_PIN), and four LEDs are connected to pins 3, 4, 5, and 6 (LED1, LED2, LED3, LED4).
- The setup() function runs once when the Arduino starts. It sets the IR sensor as INPUT and all LED pins as OUTPUT using pinMode(). Initially, all LEDs are turned OFF.
- The loop() function runs continuously. It reads the IR sensor value using digitalRead() and compares it with the previous state stored in lastIRState.
- When an object is detected (change from HIGH to LOW), the count value increases by 1. Based on the count, LEDs turn ON one by one:
- 1 → LED1 ON
- 2 → LED2 ON
- 3 → LED3 ON
- 4 → LED4 ON
- After the fourth detection, all LEDs turn OFF and the count resets to 0.
- A small delay() is added to prevent multiple detections.

So overall, each object detection lights up LEDs one by one, and then resets after all LEDs are ON.

Try This Yourself Code

This Try This Yourself code keeps the same topic of Intelligent Object Monitor but changes the pattern, values, or outputs slightly so students can practice the same idea with more confidence.

Try This Yourself Code

```
#define IR_PIN 2        // IR sensor connected to digital pin 2
#define BUZZER_PIN 5    // Buzzer connected to digital pin 5

int count = 0;          // Variable to store number of detections
int lastIRState = HIGH; // Store previous IR state to detect new object

void setup() {
  pinMode(IR_PIN, INPUT);        // Set IR sensor pin as input
  pinMode(BUZZER_PIN, OUTPUT);   // Set buzzer pin as output

  digitalWrite(BUZZER_PIN, LOW); // Keep buzzer OFF initially
}

// Function to beep 'n' times
void beep(int times) {
  for (int i = 0; i < times; i++) {     // Loop 'times' number of beeps
    digitalWrite(BUZZER_PIN, HIGH);     // Turn buzzer ON
    delay(200);                         // Wait 200 ms
    digitalWrite(BUZZER_PIN, LOW);      // Turn buzzer OFF
    delay(200);                         // Gap between beeps
  }
}

void loop() {
  int irState = digitalRead(IR_PIN); // Read IR sensor value

  // Check if object is detected (LOW) and previous state was HIGH
  if (irState == LOW && lastIRState == HIGH) {
    count++;                        // Increase detection count

    if (count >= 1 && count <= 4) {
      beep(count);                  // Beep equal to count number
    } else {
      count = 0;                    // Reset after 4 detections
    }
```

```
 delay(200); // Debounce delay to avoid multiple counts
 }

 lastIRState = irState; // Store current state for next loop
}
```

Objective:

- students practice the same topic with a small variation instead of starting from zero
- students compare the first code and the Try This Yourself code to understand what changed and why
- students build confidence by modifying a familiar circuit and logic flow

What Students Learn:
the basic idea of Intelligent Object Monitor how the code connects with the circuit behavior how to upload and test a simple Arduino sketch

Learning Outcomes
After completing this session, students should be able to:

- Understand how IR sensors monitor the presence of objects and trigger outputs for continuous observation and detection.
- Learn how sensors detect obstacles and generate alerts like LEDs or alarms for safety and protection systems.
- Understand how infrared sensors detect objects using signal reflection and control outputs in automation and robotics.
- Learn how sensor inputs are processed to create smart systems that monitor and respond to object presence automatically.

Try Out Other Things

- use the Try This Yourself code after each main class code before asking students to build a completely new variation
- ask students to point out the exact lines that changed between the first code and the Try This Yourself code
- let students first run the Try This Yourself code code as it is and then change one value, one condition, or one output